

Secure Data Protection using AES-GCM and ChaCha20-Poly1305

A design-decision reference for the sender/receiver AEAD subsystem — what was built, the reasoning behind each choice, and the alternatives that were on the table but not taken.

Design notes & alternatives — reference material for the Assignment 1 Report and viva.

CONTENTS

01 What this project is	09 Associated Data design
02 Architecture at a glance	10 Replay protection
03 End-to-end data flow	11 Authentication-failure handling
04 AEAD from first principles	12 Key management (out of scope)
05 AES-128-GCM internals	13 Testing strategy
06 ChaCha20-Poly1305 internals	14 Performance results
07 Wire format design	15 Requirement coverage map
08 Nonce management	16 Limitations & what's next

01 What this project is

The codebase simulates two parties — a `Sender` and a `Receiver` — exchanging small structured messages (`Record`s) over a channel that cannot be trusted to preserve confidentiality or integrity. Rather than inventing a cryptographic primitive, the assignment's point is **secure engineering**: correctly composing an existing AEAD primitive (AES-128-GCM or ChaCha20-Poly1305, both via OpenSSL's EVP API) into a protocol that also handles nonce management, associated data, and replay — the parts that don't come for free just because you called `EVP_EncryptUpdate`.

WHY IT MATTERS Most real-world crypto failures aren't broken ciphers — they're broken protocol logic around a correct cipher: reused nonces, unauthenticated headers, replayed messages, or errors that leak plaintext before verification finishes. This project's structure exists specifically to force those decisions into the open.

02 Architecture at a glance

Five small classes, each owning exactly one concern, wired together by two orchestrators.

CLASS	OWNS	DELIBERATELY DOES <i>NOT</i> KNOW ABOUT
<code>Record</code>	The message shape: id, timestamp, payload	Bytes, encryption, files
<code>Serializer</code>	Fixed-width <code>Record</code> $\rightleftarrows$ byte encoding	Cryptography
<code>Crypt</code>	AEAD engine — wire format, nonce generation, encrypt/decrypt for both algorithms	<code>Record</code> , files, the concept of a "sender"
<code>Sender</code>	Build record $\rightarrow$ serialize $\rightarrow$ encrypt $\rightarrow$ write channel	How encryption or serialization work internally
<code>Receiver</code>	Read channel $\rightarrow$ decrypt $\rightarrow$ verify $\rightarrow$ replay-check $\rightarrow$ deserialize	How encryption or serialization work internally

WHY THIS LAYERING The alternative — one class that reads a string, encrypts it, and writes a file — would be shorter, but it collapses "what is a message" (a domain concept) into "how is it protected" (a cryptography concept). Keeping `Crypt` ignorant of `Record` means the AEAD engine is independently testable and reusable for any byte blob, which is exactly what `test_suite.cpp` exploits by calling `Crypt::Encrypt / Decrypt` directly for the performance benchmarks, bypassing serialization entirely.

03 End-to-end data flow

What happens between `sender.Send("hello")` and the string reappearing at `receiver.Receive()`.

1 Build the record.

```
Sender::CreateRecord
```

assigns the next sequence id (a process-wide monotonic counter) and the current timestamp.

```
Record{ recId, timestamp, data }
```

2 Serialize.

```
Serializer::Serialize
```

packs

```
timestamp
```

and

```
data
```

into a fixed-width byte layout — *not* a raw

```
memcpy
```

of the struct, because struct layout isn't portable across compilers/platforms.

```
[8B timestamp][4B length][N bytes payload]
```

3 Split off the AAD. The record id is *not* serialized into the payload — it's encoded separately via

```
GetAADBytes()
```

and fed to the cipher as Associated Data instead of as plaintext.

4 **Encrypt.**

```
Crypt::Encrypt
```

generates a fresh random nonce, runs the selected AEAD cipher over the serialized bytes with the AAD bound in, and returns ciphertext + tag.

5 **Frame the wire format.** The four artifacts are concatenated into one blob.

```
secureStream = AAD(4B) || nonce(12B) || ciphertext || tag(16B)
```

6 **Cross the channel.**

```
Sender
```

writes the blob to

```
tm.bin
```

— standing in for a socket, queue, or disk handoff.

7 **Parse the wire format.**

```
Crypt::Decrypt
```

slices the blob back into AAD / nonce / ciphertext / tag using the same fixed offsets.

8 **Decrypt + authenticate.** The cipher recomputes the tag over the ciphertext and AAD and compares it to the received tag *before* releasing any plaintext. Mismatch → exception, nothing returned.

9 **Check for replay.**

```
Receiver
```

decodes the recId from the now-trusted AAD and rejects it if that id has been accepted before.

10 **Deserialize.**

```
Serializer::Deserialize
```

turns the verified plaintext back into a

```
Record
```

, which

```
Receiver::Receive
```

hands back as a plain string.

04 AEAD, from first principles

Why the assignment insists on AEAD specifically, rather than "encryption" in general.

Plain encryption (e.g. AES-CTR alone) gives confidentiality but not integrity — an attacker who can't read the plaintext can still flip bits in the ciphertext and produce a predictably corrupted plaintext on the other end, which is exactly how bit-flipping attacks against unauthenticated stream ciphers work. The historical fix was to bolt a MAC onto an encryption scheme, and there are three ways to combine them:

Composition strategies for confidentiality + integrity

SCHEME	MAC COMPUTED OVER	KNOWN WEAKNESS
Encrypt-and-MAC	Plaintext	MAC can leak plaintext info (used in SSH); doesn't guarantee ciphertext integrity
MAC-then-Encrypt	Plaintext, then MAC+plaintext both encrypted	Padding-oracle attacks (the class of bug that broke early TLS/SSL CBC modes)
Encrypt-then-MAC	Ciphertext	Provably secure if done correctly, but easy to implement wrong (nonce reuse, MAC omitted on associated data)
AEAD (GCM / Poly1305)	Ciphertext <i>and</i> unencrypted associated data, as one atomic primitive	Still fails badly on nonce reuse — but there's only one primitive to get right, not two composed independently

WHY AEAD WON AEAD collapses "encrypt correctly" and "MAC correctly, over the right data, without leaking a timing oracle" into a single library call, which is precisely what the assignment brief means by "the emphasis is on correctly integrating these algorithms" rather than composing primitives by hand.

05 AES-128-GCM internals

What `EVP_aes_128_gcm()` is actually doing.

GCM (Galois/Counter Mode) is AES run in **CTR mode** for encryption — the block cipher generates a keystream by encrypting a counter derived from the nonce, and that keystream is XORed with plaintext, making it a stream cipher built from a block cipher. Authentication is layered on top via **GHASH**, a universal hash function over $GF(2^{128})$ that processes the AAD and ciphertext to produce the tag. This is why the OpenSSL calls for AES-GCM look almost identical to a stream cipher API

(`EVP_EncryptUpdate` for data, a separate `_ctrl` call to fetch the tag) rather than a block-padding API.

Alternative AES-based AEAD modes not used here

MODE	TRADEOFF VS GCM
GCM (chosen)	Hardware-accelerated on any CPU with AES-NI + PCLMULQDQ; the de facto default (TLS 1.3, IPsec, SSH)
CCM	No hardware GHASH acceleration; two passes over the data (CBC-MAC then CTR) — used mainly in constrained IoT (802.15.4/Zigbee)
SIV (Synthetic IV)	Nonce- <i>misuse resistant</i> — safe even if a nonce repeats — but requires two passes and isn't part of the standard EVP one-shot streaming API
AES-CBC + HMAC (manual)	What GCM replaced; requires the encrypt-then-MAC discipline above, and CBC needs padding, opening padding-oracle risk

06 ChaCha20-Poly1305 internals

Why the assignment asks for a second, structurally different algorithm rather than a second AES mode.

ChaCha20 is a pure software stream cipher — it never needs a lookup table or dedicated silicon to run at speed, because it's built entirely from add-rotate-XOR (ARX) operations on 32-bit words. That also makes it inherently free of the cache-timing side channels that plagued naive AES table-lookup implementations before AES-NI existed. Poly1305 is a polynomial MAC evaluated in $GF(2^{130}-5)$ rather than $GF(2^{128})$ — a completely different mathematical construction from GHASH, computed once per message from a one-time key derived from the ChaCha20 keystream's first block (RFC 8439's construction, which is what OpenSSL's `EVP_chacha20_poly1305()` implements).

WHY IT'S THE SECOND ALGORITHM HERE The assignment wants a genuine comparison, not two flavors of the same design. AES-GCM depends on hardware acceleration to be fast and constant-time; ChaCha20-Poly1305 is fast and constant-time *everywhere*, including phones and embedded cores without AES-NI. That's a real, measurable engineering tradeoff — see §14 — not just a policy choice.

07 Wire format design

Why `AAD || nonce || ciphertext || tag`, concatenated with fixed widths, instead of something more self-describing.

Offset	0	4	16	len-16	len
	+-----+-----+-----+-----+				
	AAD	nonce	ciphertext	tag	
	4 bytes	12 bytes	variable	16 bytes	
	+-----+-----+-----+-----+				

Framing alternatives considered

APPROACH	TRADEOFF
Fixed-width concatenation (chosen)	Zero parsing ambiguity, no extra bytes spent on length prefixes — but only works because every field <i>except</i> ciphertext has a length fixed by the algorithm choice
Length-prefixed TLV	Self-describing and extensible (can add fields later without breaking old parsers) — costs a few bytes per record and a bit more parsing code; the standard choice for a real wire protocol
Structured envelope (protobuf / JSON+base64)	Versionable and language-agnostic, natural if sender/receiver were separate services — pure overhead for a single-process file handoff
Out-of-band header	AAD/nonce sent via a separate channel or fixed protocol field (this is what TLS records and IPsec ESP actually do) — avoids re-deriving offsets, but there's no "protocol field" to reuse here

The fixed-width scheme is only safe because the ciphertext is the *last* variable-length field before a fixed-length tag — if two variable-length fields existed, positional slicing would be ambiguous and a length prefix would become mandatory.

08 Nonce management

FR-5 / SR-3: *correctly manage nonces, and never reuse one with the same key.*

`Crypt::GenerateNonce` draws a fresh 96-bit value from `RAND_bytes` (OpenSSL's CSPRNG) for every single call to `Encrypt` — nothing is cached or derived from a counter. This is the standard, simplest-to-get-right nonce strategy for both GCM and ChaCha20-Poly1305, which is why both ciphers standardize on a 96-bit nonce in the first place.

Nonce strategies compared

STRATEGY	REUSE RISK	COST
Random 96-bit (chosen)	Birthday-bound collision risk grows past $\approx 2^{32}$ messages under one key	None — no shared state between sender calls
Monotonic counter	Zero collision risk by construction, <i>if</i> state is durable	Sender must persist the counter across restarts, or a crash-and-reuse bug silently breaks security
XChaCha20-Poly1305 (192-bit nonce)	Random nonces become safe up to $\approx 2^{96}$ messages — the birthday bound moves out of reach	Not in the assignment's two required configurations; would need a different EVP cipher (<code>EVP_chacha20_poly1305</code> doesn't have an X-variant in stock OpenSSL 3.x)
Partitioned counter (multi-sender)	Zero risk across senders sharing a key, via a fixed sender-id prefix + local counter	Extra coordination/allocation of the id space; the pattern used by QUIC and IPsec SAs

WHY RANDOM OVER COUNTER A counter is safer in the limit but only if it survives a crash — get that wrong and you get catastrophic, silent nonce reuse. Random nonces degrade gracefully (the risk is probabilistic and bounded, not a hard failure mode) and need zero persisted state, which matches the assignment's message volume (TR-7 asks for ~10,000 records — nowhere near the 2^{32} birthday bound) far better than the operational complexity a durable counter would add for a demo subsystem.

`test_suite.cpp`'s TR-7 test empirically checks this: it sends 10,000 records under one key and collects every nonce into a `std::set`, asserting the set's size stays at 10,000 (i.e., zero collisions observed).

09 Associated Data design

FR-4 / SR-4: *what gets authenticated but not encrypted, and why it's just the record id.*

The 4-byte record id is bound into the tag as AAD via `Serializer::GetAADBytes`. It stays visible in plaintext on the wire (record ids usually aren't secret) but can't be detached from its ciphertext or swapped for another record's id without invalidating the tag.

What could have been chosen as AAD

AAD CONTENT	WHAT IT PROTECTS	COST
Record id only (chosen)	Binds ciphertext to its sequence position — enables replay detection	Minimal; 4 bytes
No AAD	Nothing beyond the payload itself	Cheapest, but nothing stops a valid ciphertext from being replayed under a different apparent sequence number
Full header (id + timestamp + sender identity)	Also authenticates freshness and origin, closer to what a real transport protocol (e.g. a TLS record header) authenticates	More AAD bytes to transmit in the clear; timestamp freshness checks need clock-sync assumptions this assignment explicitly places out of scope
Application-level context (e.g. session id, channel id)	Prevents cross-context ciphertext splicing when one key is shared across multiple logical streams	Not applicable — this design uses one key per <code>Crypt</code> instance / one logical channel

10 Replay protection

FR-8 / SR-5 / TR-5 — *the piece that was entirely missing before this session's changes.*

`Receiver` keeps a `std::set<uint32_t> seenRecIds`. After a record decrypts and authenticates, its id (recovered from the now-trusted AAD) is checked against the set; a repeat is rejected before `Receive()` ever returns data.

Replay-detection strategies compared

STRATEGY	HANDLES OUT-OF-ORDER DELIVERY?	MEMORY PROFILE	REAL-WORLD ANALOGUE
Seen-id set (chosen)	Yes — any id, any order, exactly once	Unbounded — grows with every accepted record for the session's lifetime	—
Monotonic counter (reject id $\leq$ last seen)	No — legitimately reordered records are also rejected	$O(1)$ — a single integer	Not standard alone; usually paired with a window (below)
Sliding window (accept a bounded range, bitmap for the window)	Yes, within the window width	$O(\text{window size})$ — bounded and small	TLS 1.3 record replay window; IPsec ESP anti-replay window
Timestamp freshness (reject if outside $\pm\Delta$ of now)	Yes, within Δ	$O(1)$ if combined with a short-lived seen-cache	Kerberos ticket freshness; needs synchronized clocks

WHY AN UNBOUNDED SET, HERE

A sliding window is the production-grade answer — it's what TLS and IPsec actually ship, because an unbounded set is a memory-exhaustion liability over a long-lived connection. It was skipped here because the assignment's scope is a session-bounded demonstration (thousands of records, not an always-on service), and a plain set makes the *correctness property* — "no id is ever accepted twice" — visually obvious in the code and trivial to verify in TR-5, rather than being an artifact of a window size choice.

WORTH FLAGGING IN THE REPORT

This is a legitimate scoping tradeoff to name explicitly in the Assignment 1 Report's Discussion section — "unbounded seen-set, sufficient for this demonstration's message volume; a sliding window would be required for a long-running service" is exactly the kind of documented assumption FR-10 asks for.

11 Authentication-failure handling

FR-7 / SR-6 — reject bad records without ever releasing recovered plaintext.

OpenSSL's decrypt API is asymmetric on purpose: `EVP_DecryptUpdate` will happily emit *some* bytes even for a tampered ciphertext — the tag check only happens at `EVP_DecryptFinal_ex`, which returns ≤ 0 on mismatch. `Crypt::Decrypt` treats that as the single point of truth and throws `std::runtime_error` rather than returning a flag, which forces every caller up the chain to either handle the failure explicitly or let the record be dropped — there's no code path where a caller can accidentally read the reference-passed `plainText` buffer after a failed verification, because the exception unwinds before `Receiver::ConstructRecord` reaches the line that would use it.

WHY EXCEPTIONS OVER A RETURN CODE

A boolean/error-code return is easy to silently ignore (`if (ok) { ... }` without an `else` is a compiling, plausible-looking bug). An uncaught exception crashes loudly; a caught one forces a conscious decision at the catch site. `Receiver::ConstructRecord`'s `catch (...)` is that conscious decision — "any failure in this block means reject the record," which is the fail-closed posture SR-6 requires.

12 Key management — explicitly out of scope

The assignment states the sender and receiver already share a secret key; this project reads that key from a file.

`keys/aeskey.bin` (16 bytes) and `keys/ccpkey.bin` (32 bytes) are read raw from disk by `main.cpp`'s `GetKey`. For context — not because it needed implementing here — real systems get that shared key one of a few ways:

Pre-shared key (used here)

Provisioned out-of-band before any communication. Simple, but doesn't scale past a small, fixed set of parties and has no forward secrecy if the key ever leaks.

Key exchange (Diffie–Hellman / X3DH)

Derives a fresh shared secret per session over an insecure channel. What TLS and Signal actually do; explicitly named as a later-course topic in the assignment brief.

KDF from a master secret

One long-term key plus `HKDF` derives per-purpose subkeys (e.g. separate keys for AES-GCM vs ChaCha20 configurations) — avoids ever reusing raw key material across algorithms.

KMS / HSM-backed key

Key material never leaves a hardware boundary; the application calls an API to encrypt/decrypt rather than holding raw bytes. Standard for production cloud services.

13 Testing strategy

How `test_suite.cpp` demonstrates TR-1 through TR-8 without a real network.

The assignment allows the three logical roles (sender, malicious actor, receiver) to live inside one process, "provided that all Functional and Testing Requirements are satisfied." `test_suite.cpp` takes that option: the "malicious actor" is a function that opens `tm.bin` after a legitimate `Send()` and flips one byte at a computed offset — position `16` for ciphertext (TR-2), the final byte for the tag (TR-3), byte `0` for the AAD (TR-4).

Testing Requirement → mechanism

TR	MECHANISM	PASS CONDITION
TR-1	Plain send + receive	Recovered string equals the original
TR-2	Flip byte 16 (first ciphertext byte)	<code>Receive()</code> returns empty
TR-3	Flip the last byte (inside the tag)	<code>Receive()</code> returns empty
TR-4	Flip byte 0 (inside the AAD)	<code>Receive()</code> returns empty
TR-5	Call <code>Receive()</code> twice on one send	First call succeeds, second is rejected as a replay
TR-6	Decrypt with a same-length, different-content key	<code>Receive()</code> returns empty
TR-7	10,000 sends under one key; collect nonces into a <code>std::set</code>	Set size stays at 10,000 (no collisions)
TR-8	Time <code>Crypt::Encrypt</code> / <code>Decrypt</code> directly at 64B/1KiB/64KiB	Comparative timing table (not pass/fail)

WHY BYTE-FLIP A FILE INSTEAD OF A REAL MITM A genuine network-level attacker would intercept a socket write; flipping a byte in the shared file achieves the identical adversary model — "the attacker can modify bytes in transit" — without the assignment's added complexity of standing up sockets or processes, which the brief explicitly says isn't required ("network communication is optional").

SCOPING NOTE FOR TR-8 Performance is measured on `Crypt::Encrypt` / `Decrypt` directly, bypassing `Sender` / `Receiver`'s file I/O. Going through the full file-based pipeline would work too, but disk I/O time would dominate and mask the actual cost difference between the two ciphers — worth stating as an assumption in the report.

14 Performance results

TR-8, run locally via `make run-tests`. Numbers are one sample run on this machine — re-run for the report, hardware varies.

Encrypt/decrypt cost by payload size

ALGORITHM	SIZE	ENCRYPT	DECRYPT	ENC THROUGHPUT	DEC THROUGHPUT
AES-128-GCM	64 B	2.87 μ s	1.38 μ s	21.3 MB/s	44.1 MB/s
ChaCha20-Poly1305	64 B	2.39 μ s	1.41 μ s	25.5 MB/s	43.3 MB/s
AES-128-GCM	1 KiB	3.33 μ s	1.62 μ s	293.1 MB/s	602.9 MB/s
ChaCha20-Poly1305	1 KiB	4.11 μ s	3.27 μ s	237.6 MB/s	298.3 MB/s
AES-128-GCM	64 KiB	66.4 μ s	34.3 μ s	940.9 MB/s	1824.4 MB/s
ChaCha20-Poly1305	64 KiB	132.3 μ s	75.3 μ s	472.6 MB/s	829.6 MB/s

At 64 bytes the two are within noise of each other — fixed per-call overhead (context setup, a handful of AES/ChaCha rounds) dominates at that size, not the cipher's steady-state throughput. By 1 KiB and clearly by 64 KiB, AES-128-GCM pulls roughly 2 $\times$ ahead.

WHY AES-GCM WINS ON THIS MACHINE

This CPU has **AES-NI** and **PCLMULQDQ** instructions — dedicated silicon that does one AES round or one GHASH multiply per instruction. ChaCha20-Poly1305 has no equivalent hardware fast path; it runs as general-purpose ARX/polynomial arithmetic on the ALU regardless of what CPU it's on. That's precisely the tradeoff named in §6: on a CPU *without* AES-NI — many low-end ARM cores, some older mobile chips, some embedded/IoT targets — this result inverts and ChaCha20-Poly1305 wins, which is the whole reason it exists as a TLS 1.3 cipher suite alongside AES-GCM rather than as a replacement for it.

15 Requirement coverage map

REQUIREMENT	STATEMENT	WHERE
FR-1..3, FR-9	Record protection, both AEAD configs, equivalent behavior	<code>crypt.{h,cpp}</code>
FR-4 / SR-4	AAD authenticated and verified	<code>serializer.cpp</code> (<code>GetAADBytes</code>), <code>crypt.cpp</code> (<code>Decrypt</code>)
FR-5 / SR-3	Nonce management, no reuse under normal operation	<code>crypt.cpp</code> (<code>GenerateNonce</code>); verified by TR-7
FR-6 / SR-2	Correct record recovery + integrity detection	<code>crypt.cpp</code> (<code>Decrypt</code> , <code>DecryptAES_GCM</code> , <code>DecryptChaCha_Poly</code>)
FR-7 / SR-6	Reject on authentication failure, release nothing	<code>receiver.cpp</code> (<code>catch</code> in <code>ConstructRecord</code>)
FR-8 / SR-5	Replay detection	<code>receiver.{h,cpp}</code> (<code>seenRecIds</code>)
SR-1	Confidentiality	AEAD encryption itself
TR-1..8	Demonstrable, evidenced tests	<code>test_suite.cpp</code>
D3	README with build/run instructions	<code>README.md</code>

16 Limitations & what production hardening would add

- **Unbounded replay set** — see §10; a sliding window bounds memory for long-lived sessions.
- **No key rotation or expiry** — a static key file has no mechanism to retire compromised key material.
- **No transport authentication of the channel itself** — `tm.bin` has no access control; a real deployment needs the channel (socket, queue) secured or the threat model explicitly scoped to "ciphertext-only visibility," which the AEAD tag already defends against.
- **Single-threaded, single-session assumptions** — `Sender::sendCount` is a process-wide static counter; a multi-sender system sharing one key would need the partitioned-nonce scheme from §8 to keep nonce spaces disjoint.

None of these are bugs relative to the assignment's stated scope (§3.2 of the brief explicitly excludes key exchange, PKI, and transport-layer protocols) — they're the documented assumptions the Assignment 1 Report's Discussion section is asking for.